8-BIT OPERATIONS (WITH COMMENTS)

```
ORG 100H             ; ORIGIN FOR .COM PROGRAM

; STORE OPERAND1 AND OPERAND2
MOV AL, 21H          ; OPERAND1 = 33 DECIMAL
MOV BL, 1AH          ; OPERAND2 = 26 DECIMAL

; -------- ADDITION: AL + BL --------
MOV CL, AL           ; COPY AL TO CL
ADD CL, BL           ; CL = AL + BL
MOV [0200H], CL      ; STORE RESULT AT MEMORY LOCATION 0200H

; -------- SUBTRACTION: AL - BL --------
MOV CL, AL           ; COPY AL AGAIN
SUB CL, BL           ; CL = AL - BL
MOV [0201H], CL      ; STORE RESULT AT 0201H

; -------- INCREMENT: AL + 1 --------
MOV CL, AL
INC CL               ; INCREMENT BY 1
MOV [0202H], CL      ; STORE RESULT AT 0202H

; -------- DECREMENT: BL - 1 --------
MOV CL, BL
DEC CL               ; DECREMENT BY 1
MOV [0203H], CL      ; STORE RESULT AT 0203H

RET
```

MULTIBYTE ADD/SUB (WITH COMMENTS)

```
ORG 100H

; ------------------ DATA (STORE NUMBERS) ------------------
; STORE FIRST 4-BYTE NUMBER AT 3000H (LITTLE-ENDIAN)
MOV SI, 3000H
MOV [SI], 34H        ; BYTE0 (LSB)
MOV [SI+1], 12H      ; BYTE1
MOV [SI+2], 00H      ; BYTE2
MOV [SI+3], 00H      ; BYTE3 (MSB)

; STORE SECOND 4-BYTE NUMBER AT 3010H
MOV SI, 3010H
MOV [SI], 10H        ; BYTE0
MOV [SI+1], 02H      ; BYTE1
MOV [SI+2], 00H
MOV [SI+3], 00H

; ------------------ MULTI-BYTE ADDITION ------------------
MOV SI, 3000H        ; OPERAND1
MOV DI, 3010H        ; OPERAND2
MOV BX, 3020H        ; RESULT
MOV CX, 4            ; FOUR BYTES
CLC                  ; CLEAR CARRY

ADD_LOOP:
MOV AL, [SI]
ADC AL, [DI]         ; ADD WITH CARRY
MOV [BX], AL
INC SI
INC DI
INC BX
LOOP ADD_LOOP

; ------------------ MULTI-BYTE SUBTRACTION ------------------
MOV SI, 3000H
MOV DI, 3010H
MOV BX, 3030H        ; RESULT FOR SUBTRACTION
MOV CX, 4
```

```
CLC                 ; CLEAR BORROW

SUB_LOOP:
MOV AL, [SI]
SBB AL, [DI]        ; SUBTRACT WITH BORROW
MOV [BX], AL
INC SI
INC DI
INC BX
LOOP SUB_LOOP

RET
```

ASCENDING SORT (WITH COMMENTS)

```
MODEL SMALL
STACK 100H

DATA SEGMENT
ARR DB 34, 12, 56, 9, 21      ; 5 ELEMENTS TO SORT
N   DB 5
DATA ENDS

CODE SEGMENT
MAIN:
MOV CX, 4                     ; OUTER LOOP = N - 1

OUTER_LOOP:
MOV SI, 0                     ; INDEX = 0
MOV BX, CX                    ; INNER LOOP COUNT

INNER_LOOP:
MOV AL, ARR[SI]
CMP AL, ARR[SI+1]             ; COMPARE ADJACENT ELEMENTS
JBE NO_SWAP                   ; IF AL <= NEXT → NO SWAP

; ----- SWAP -----
MOV AH, ARR[SI+1]
MOV ARR[SI], AH
MOV ARR[SI+1], AL

NO_SWAP:
INC SI
DEC BX
JNZ INNER_LOOP

DEC CX
JNZ OUTER_LOOP

MOV AH, 4CH
INT 21H
CODE ENDS
END MAIN
```

MATRIX ADDITION (WITH COMMENTS)

```
MODEL SMALL
STACK 100H

DATA SEGMENT
MATRIXA DB 1,2,3,4,5,6,7,8,9
MATRIXB DB 9,8,7,6,5,4,3,2,1
MATRIXADD DB 9 DUP(0)
DATA ENDS

CODE SEGMENT
MAIN:
MOV AX, @DATA
```

```
MOV DS, AX

; -------- MATRIX ADDITION A + B --------
LEA SI, MATRIXA
LEA DI, MATRIXB
LEA BX, MATRIXADD
MOV CX, 9

ADD_LOOP:
MOV AL, [SI]
ADD AL, [DI]
MOV [BX], AL
INC SI
INC DI
INC BX
LOOP ADD_LOOP

MOV AH, 4CH
INT 21H
CODE ENDS
END MAIN
```

SQUARE & CUBE (WITH COMMENTS)

```
MODEL SMALL
STACK 100H

DATA SEGMENT
NUM DB 5                ; NUMBER TO SQUARE AND CUBE
SQUARE DW 0
CUBE DW 0
DATA ENDS

CODE SEGMENT
MAIN:
MOV AX, @DATA
MOV DS, AX

MOV AL, NUM
MOV AH, 0
MOV BL, AL
MUL BL                  ; AL * AL = AX
MOV SQUARE, AX

MOV AX, SQUARE
MOV BL, NUM
MUL BL                  ; SQUARE * NUM = CUBE
MOV CUBE, AX

MOV AH, 4CH
INT 21H
CODE ENDS
END MAIN
```